

Introduccion al Lenguaje Z

Contenido

- **Descripción**
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Descripcion

- El Lenguaje Z es un lenguaje utilizado en Ingenieria del software para la ***especificación formal de un*** sistema de computo, como una fase previa al desarrollo del codigo de programa para el mismo en un lenguaje de programacion.
- Desarrollado por Jean-Raymond Abrial mientras formaba parte del Grupo de investigacion en Programacion del Laboratorio de computacion de la Universidad de Oxford.
- El lenguaje Z se basa en la teoria de conjuntos, el calculo lambda y la logica de primer orden.
- *Se definen construcciones denominadas esquemas para describir el espacio de estados del sistema y las operaciones que sobre el mismo se efectuan. En los esquemas se declaran variables y predicados que afectan los valores de las variables declaradas.*

Ejemplo: Sistema que registra los cumpleaños de las personas

- Necesitamos conocer el nombre de las personas y su fecha de cumpleaños
- Tipos basicos de datos:
 - [NOMBRE, FECHA]
- Primero: Describir su “espacio de estado ” (esquema)

$[NOMBRE, FECHA]$

AgendaCumple

contactos : $\mathbb{P} NOMBRE$

cumple: $NOMBRE \rightarrow FECHA$

contactos = $\text{dom } \textit{cumple}$

Invariante del sistema

Posible estado:

- Tres personas en el conjunto nombres conocidos (***known***), con sus respectivas fechas de cumpleaños registrados por la función ***cumpleaños (birthday)***:

$known = \{ \text{John, Mike, Susan} \}$

$birthday = \{ \begin{array}{l} \text{John} \mapsto \text{25-Mar}, \\ \text{Mike} \mapsto \text{20-Dec}, \\ \text{Susan} \mapsto \text{20-Dec} \end{array} \}.$

- El invariante se satisface, porque ***birthday*** registra una fecha de exactamente de los tres nombres contenidos en nombres conocidos ***known***.

Agregar un nuevo cumpleaños

AgregarCumpleaños

Δ AgendaCumple

nombre? : NOMBRE

fecha? : FECHA

nobre? $\notin$ conocidos

cumpleaños' == cumpleaños $\cup$ { nombre $\rightarrow$ fecha }

Contenido

- Descripción
- **Conjuntos**
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Conjuntos

- Un *objeto* es cualquier cosa que se pueda ver, tocar, oler o pensar, etc. Una coleccion de objetos similares es llamado un *conjunto*.
- *Ejemplos:*
 - La coleccion de personas que inscribieron el curso de M.F.
 - La coleccion de anos bisiestos desde 1986.
 - La coleccion de passwords de tres letras que se pueden formar con las letras a, b y c.
 - La coleccion de numeros enteros positivos

Nombre de conjunto

- Si un conjunto tienen muchos elementos, le damos un nombre que lo describa.
- Por ejemplo:
 - *PERSONA* – Conjunto de todas las personas
 - *ANOBISIESTO* – Conjunto de todos los años bisiestos pasados
 - *PASSWORD* – Conjunto de todas las posibles passwords
- Convencion
 - Son escrito con mayuscula
 - Son singulares
 - No contiene espacios, underscores, puntos, tec.

ConjuntosTipicos

- Algunos conjuntos estandares.
 - **Z** El conjunto de todos los numeros enteros (no confundir con la notacion Z)
 - **N** El conjunto de todos los numeros naturales
 - **$\emptyset$** El conjunto vacio
- Precisiones:
 - **$\emptyset$** es subconjunto de cualquier conjunto
 - **N** es subconjunto de Z

Cuidado con la notacion de conjunto en Z

- El conjunto de todos los elementos de A que satisfacen la condición $P(x)$
 - $\{x : A \mid P(x)\}$
 - Nótese que:
 - $x \in A \equiv x : A$
 - La $\mid$ es usada para indicar un predicado
- Z utiliza la “builder set notation”
 - $\{x : A \mid P(x) \bullet F(x)\}$
 - Conjunto de todos los valores $F(x)$ donde x pertenece a A y $P(x)$ es verdad.
- La teoría de conjuntos de Z, se basa en la axiomatización de
- Zermelo-Fraenkel

Contenido

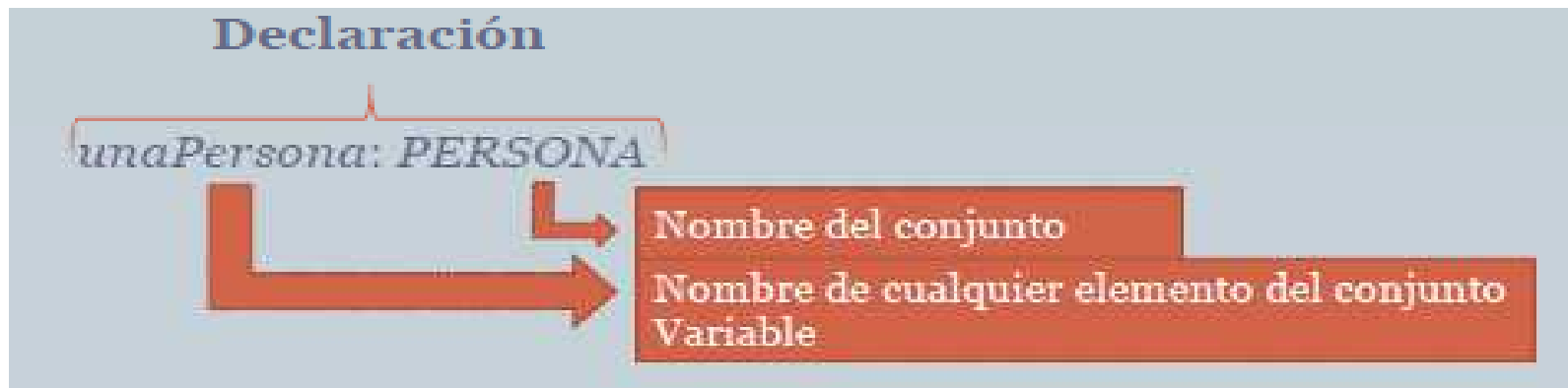
- Descripción
- Conjuntos
- **Tipos Basicos**
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Tipos Basicos. Introduccion

- Para describir el ambito en el cual trabajamos, clasificamos los objetos en conjuntos llamados **tipos**.
- El unico tipo incorporado como parte de la notacion Z, es el conjunto de los enteros, **Z**.
- Todo otro tipo es definido por nosotros.
- Para definir un *tipo basico* debemos centrarnos en lo esencial y los detalles los dejamos de lado. Esta focalizacion se denomina *abstraccion*.
- Ejemplo: Una persona tiene un nombre, una fecha de nacimiento y una direccion. Definimos los tipos:
 - [DIRECCION, FECHA, NOMBRE]
- Entre parentesis cuadrados y en orden alfabetico
- Se deben describir claramente. Por ejemplo; NOMBRE es el conjunto de todos los posibles nombre que cualquier persona pueda tener.

Declaraciones

- Sea *PERSONA* el conjunto de todas las posibles personas sobre el Universo. Si queremos referirnos a alguna en particular escribimos:



Una **variable** tiene un nombre (*unaPersona*), un tipo (*PERSONA*) y un valor extraído desde el tipo.

Declaraciones

- Ejemplo: Considere la declaracion:
 - *unaEdadPersona: 0..130*
- El nombre de la variable es “unaEdadPersona”, su tipo es **Z**, debido que los valores que puede tomar la variables son todos elementos de **Z**.
- Observacion: No puede hacer una declaracion si su tipo no ha sido definido.

Nombres de Variables

Debemos usar nombres descriptivos para denominar variables.

Por ejemplo:

- *unaEdadPersona: 0..130 OK*
- *a: 0..130 No*

Convencion:

- Siempre comenzar con una letra minuscula.
- Escribir como una combinacion de letras mayusculas y minusculas, no usar solo minusculas.
- No debe contener espacios en blanco, underscore, caracteres especiales.

Cual de las siguientes declaraciones esta bien escrita?:

- *numerodePersonas: 0..100*
- *NumerodePersonas: 0..100*
- *NUMERODEPERSONAS: 0..100*
- *n: 0..100*

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- **El Tipo Entero**
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

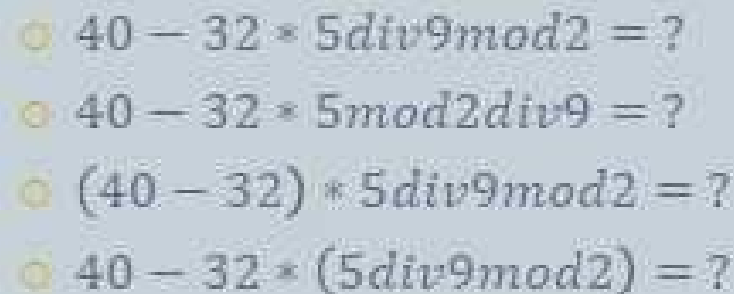
El Tipo Entero

- El tipo **Z** es el unico tipo incorporado en Z, con valores
 - ...-2, -1, 0, 1, 2,
- Operadores aritmeticos: $+$, $-$, $*$, *div* y *mod*
- Funciones: *max* y *min*
- Operadores relacionales: $>$, $<$, $\geq$ y $\leq$

Precedencias en tipo entero

- Precedencia es el orden en el cual las operaciones son llevadas a cabo.
- En aritmetica:
 - 1) Parentesis la mas alta prioridad
 - 2) *, div, y mod en el orden escrito de izquierda a derecha
 - 3) + y – en el orden escrito de izquierda a derecha

- Ejemplo:



Four mathematical expressions are listed, each preceded by a yellow circle icon. The expressions demonstrate different ways to group operations in the formula $40 - 32 * 5 \text{ div } 9 \text{ mod } 2$ to show how precedence changes with parentheses.

- $40 - 32 * 5 \text{ div } 9 \text{ mod } 2 = ?$
- $40 - 32 * 5 \text{ mod } 2 \text{ div } 9 = ?$
- $(40 - 32) * 5 \text{ div } 9 \text{ mod } 2 = ?$
- $40 - 32 * (5 \text{ div } 9 \text{ mod } 2) = ?$

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- **Predicados**
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Predicados

- Los valores de las variables pueden ser restringidos por ***predicados y al satisfacerse un predicado se define un*** conjunto.
- Los tipos de predicados son:
 - =, $\in$: igualdad y pertenencia
 - >, $\geq$, <, $\leq$: relaciones
 - $\wedge$, $\vee$: conectivos logicos
- Descripcion del nuevo conjunto: { declaracion | predicado }

- Ejemplos:

$\{ n: \mathbb{Z} \mid n > 0 \}$

$\{ n: 1..9 \mid n \bmod 2 = 0 \}$

$\{ n: \mathbb{Z} \mid n \geq 0 \wedge m > 2 \wedge n \bmod 2 \neq 0 \}$

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- **Esquemas**
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Esquemas (Schemas)

- Un esquema representa el estado de un sistema.
- Una coleccion de esquemas representa el comportamiento de un sistema computacional.
- Sistema: Desarrollar un simple *contador*.

Estado

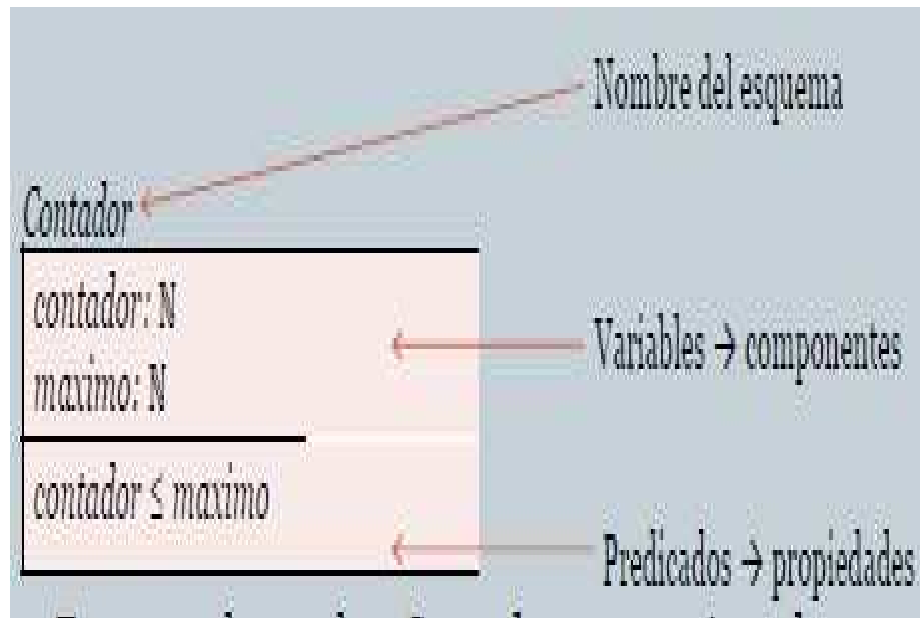
- El contenido de la memoria del sistema es su *estado*.
 - Valor actual del contador
 - Valor maximo que puede alcanzar
- Estado inicial es cuando el contador esta en cero, maximo es 9999
 - *Estado inicial: contador = 0, maximo = 9999*
- Podriamos tener un estado intermedio cuando el contador esta entre cero y el valor maximo
 - *Estado intermedio: contador = 147, maximo = 9999*
- Podriamos tener un estado final cuando el contador alcanza su valor maximo y no puede seguir avanzando
 - *Estado final: contador = 9999, maximo = 9999*

Definicion abreviada

- Contador y maximo son ambos numero. Podriamos representarlos como entero del tipo $\mathbb{Z}$, pero permitiriamos valores negativos para *contador* y para *maximo*. De modo que introducimos $\mathbb{N}$ como una abreviacion para el conjunto de todos los numero enteros excluyendo los valores negativos
 - Declarando $\mathbb{N}$ de esta forma podemos usar los operadores $<$, $>$ y $=$. El tipo de $\mathbb{N}$ es $\mathbb{Z}$.
- El simbolo $==$ se conoce como el simbolo de defincion.

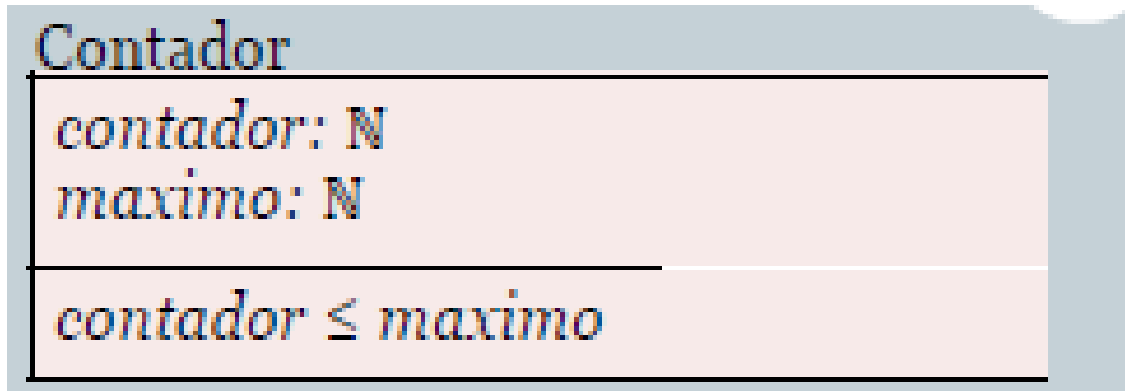
$$\mathbb{N} == \{n: \mathbb{Z} \mid n \geq 0\}$$

Esquema de Estado de Sistema



- Un *esquema* es un conjunto de variables junto con un conjunto de predicados que limitan estas variables. Se dibuja como una caja abierta.
- Esquema de nombre *Contador* que contiene dos variables, *contador* y *maximo*, y un predicado "*contador ≤ maximo*"

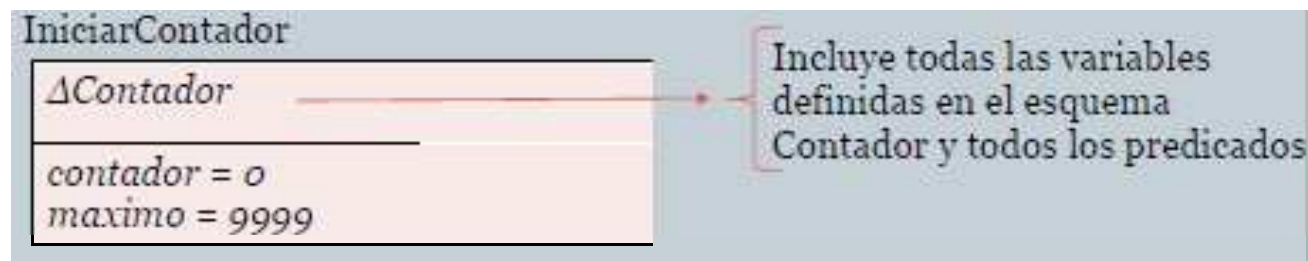
Esquema del Estado del Sistema



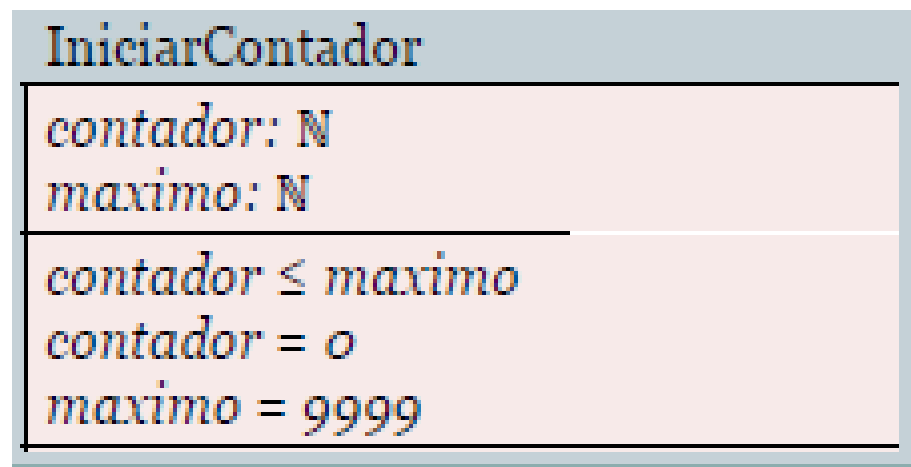
- El nombre de un esquema debe comenzar con mayuscula y no puede contener espacios en blanco.
- Si existiera mas de un predicado, estos se acoplan por medio de la conjuncion, actua por defecto.
- El esquema en que se describe las variables de un sistema y los predicados de estas variables se denomina ***esquema del estado del sistema***.
- ***Este representa el estado de un objeto***, la coleccion de valores almacenados en las variables en un instante de tiempo. Sus variables son llamadas ***variables de estado***.

Estado Inicial

- Definamos el esquema IniciarContador que describe el estado inicial de Contador.



- Equivalente a:



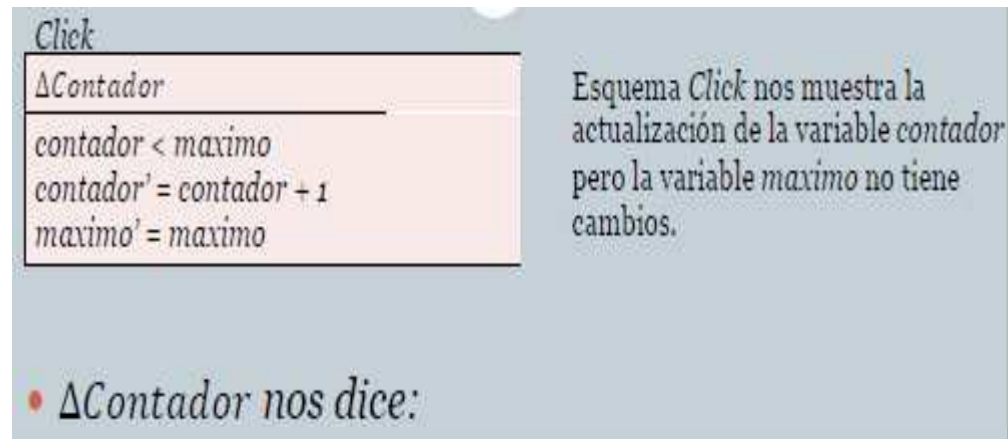
Consulta al Estado del Sistema

ConsultaContador	
$\exists \text{Contador}$	
$\text{contador!} : \mathbb{N}$	
$\text{contador!} =$	
contador	

- (Xi contador) incluye todas las variables y predicados definidos en el esquema Contador; los valores almacenados en estas variables no cambiaran.
- La declaracion ***contador! : $\mathbb{N}$*** que el valor de salida de contador pertenece al conjunto $\mathbb{N}$. Usamos el simbolo (!) para la salida.
- El predicado ***contador! = contador*** nos dice que la salida del contador es la misma que contador.

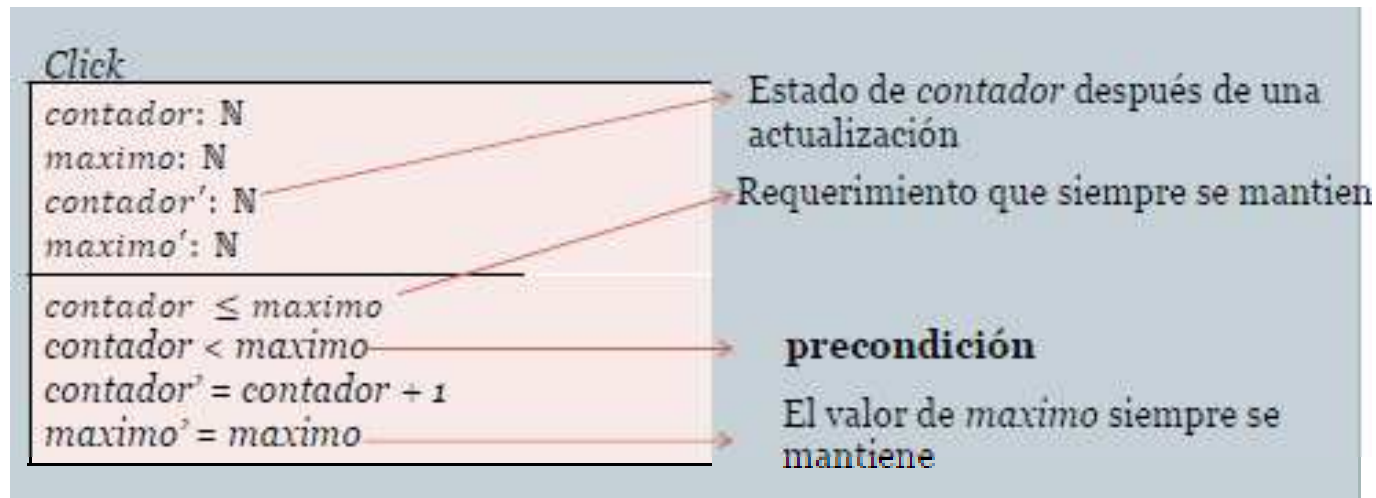
$\exists \text{Contador}$

Cambio en el estado del sistema



- Incluye todas las variables y predicados definidos en el esquema Contador.
- Los valores almacenados en alguna (o todas) de estas variables pueden cambiar.
- Toda variable seguida por el signo ' representa el estado despues de que una actualizacion se ha realizado.

Cambio en el estado del sistema. Esquema equivalente



- Una **precondición de una operacion en un esquema describe el** conjunto de estados para los cuales la salida de la operacion es definida.
- En nuestro caso, en el esquema “Click”, el conjunto de estados para los cuales la salida esta definida es:

contador ∈ 1..9999

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- **Tipos de Conjuntos**
- Operaciones entre Conjuntos
- Schema Calculus
- Relaciones Binarias

Tipo Conjunto

- Una variable de un tipo conjunto es un conjunto y es usada para representar colecciones de elementos tales como una clase de estudiantes o un conjunto de habitaciones de hotel.

- Subconjunto propio

$A \subset B$ si todo elemento de A está en B pero $A \neq B$
Ejemplo: $\mathbb{N} \subset \mathbb{Z}$

- Subconjunto

$A \subseteq B$ si todo elemento de A está en B

- Ejemplo en Z. Si tenemos la siguiente declaración:

$x: \mathbb{F}\{1, 2, 3\}$; entonces
 $x \in \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$

Declaraciones Tipo Conjunto

- Ejemplo: grupos de alumnos estudian una serie de materias.

• *ALUMNO* – El conjunto de todos los alumnos
• *MATERIA* – El conjunto de todas las materias

- Tipos:

- Podríamos definir:

• grupoAlumnos: *FALUMNO*
• grupoMaterias: *FMATERIA*

F

significa que grupoAlumnos es un subconjunto de grupo ALUMNO.

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- **Operaciones entre Conjuntos**
- Schema Calculus
- Relaciones Binarias

- Union $\rightarrow \cup$
 - $\{1, 2\} \cup \{3, 4\} = \{1, 2, 3, 4\}$
- Intersección $\rightarrow \cap$
 - $\{1, 2, 3\} \cap \{3, 4\} = \{3\}$
- Diferencia $\rightarrow \setminus$
 - $\{1, 2, 3, 4\} \setminus \{2, 4\} = \{1, 3\}$

- Precedencia
 - Paréntesis
 - Intersección
 - Unión y diferencia

- Ejemplo:
 - $\{2, 4\} \cup (\{1, 2, 3\} \setminus \{3, 4\}) \cap \{1, 3\} = ?$

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- **Schema Calculus**
- Relaciones Binarias

Calculo de Esquemas

- Anteriormente hemos visto como declaraciones y predicados fueron combinados en estructura llamadas esquemas. Vimos como un esquema puede representar el estado de un sistema, que un esquema puede cambiar el estado del sistema y una colección de esquemas modela el ambito de un sistema.
- Veremos como esquemas grandes pueden ser formados combinando esquemas mas pequenos usando conjuncion ($\wedge$) y la disyuncion $\vee$, por medio de un ejemplo.

El Sistema Clase

- Ejemplo: Los estudiantes pueden disfrutar de la clase de lenguaje Z si esta no esta completa. Aquellos que completen exitosamente todas las tareas seran recompensados con un certificado firmado por el profesor.
- Tipo basico *ALUMNO* dado por el conjunto *[ALUMNO]*
- Existe un numero limite de alumnos que pueden disfrutar la clase.

$N == \{n : \mathbb{Z} \mid n \geq 0\}$

maxTamañoClase: N

maxTamañoClase = 20

Definimos la variable global maxTamañoClase

Clase

inscrito: F ALUMNO

aprobado: F ALUMNO

#inscrito $\leq$ maxTamañoClase

aprobado $\subseteq$ inscrito

Inicialmente no existen alumnos inscritos ni alumnos aprobados

InicioClase

Clase

inscrito = $\emptyset$

aprobado = $\emptyset$

El Sistema Clase

- Un alumno se puede inscribir en la clase si esta no está completa y si no se ha inscrito con anterioridad.
- Un nuevo alumno no puede haber aprobado todos sus trabajos

BienInscrito

Δ Clase

alumno?: **ALUMNO**

$\#inscrito < maxTamañoClase$

$alumno? \notin inscrito$

$inscrito' = inscrito \cup \{alumno?\}$

$aprobado' = aprobado$

El signo ? significa entrada

El Sistema Clase

- Un alumno que ya existe es transferido al conjunto *aprobado* si ha Aprobado todos sus trabajos y no haya sido transferido previamente.
- Todo alumno en el conjunto *aprobado* debe también estar en el conjunto *inscrito*

BienCompletado

Δ Clase

alumno?: **ALUMNO**

alumno? $\in$ inscrito

alumno? $\notin$ aprobado

inscrito' = inscrito

aprobado' = aprobado $\cup$ {alumno?}

El Sistema Clase

- Solamente aquellos alumnos que han aprobado pueden obtener un *certificado*

ObtenerCertificado

Δ Clase

alumno?: ALUMNO

alumno? $\in$ aprobado

inscrito' = inscrito $\setminus$ {alumno?}

aprobado' = aprobado $\setminus$ {alumno?}

Algunos aspecto no considerados:

- La clase esta completa y nadie puede ser inscrito en ella.
- Un alumno puede ser inscrito dos veces.
- Un alumno no inscrito puede ser aprobado

Definición de Tipo Libre (FREE)

- Posible solución a temas pendientes (errores)

Dibujar una tabla de precondiciones, el conjunto de estados para los cuales las salidas exitosas están bien definidas. Posteriormente agregamos una columna las condiciones en las cuales se producen errores.

Esquema	Precondición de éxito	Condición de error
BienInscrito	$\#inscrito < \text{maxTamañoClase}$ $\text{alumno?} \in \text{inscrito}$	Clase completa: $\#inscrito \geq \text{maxTamañoClase}$ Alumno ya inscrito: $\text{alumno?} \in \text{inscrito}$
BienCompletado	$\text{alumno?} \in \text{inscrito}$ $\text{alumno?} \in \text{aprobado}$	Alumno no inscrito: $\text{alumno?} \notin \text{inscrito}$ Alumno ya aprobado: $\text{alumno?} \in \text{aprobado}$
ObtenerCertificado	$\text{alumno?} \in \text{aprobado}$	Alumno no aprobado: $\text{alumno?} \notin \text{aprobado}$

Definicion de Tipo Libre (FREE)

- En una *definicion de tipo libre*, definimos *REPORT* como el conjunto de todos los valores que describen o bien un esquema exitoso o las razones de su error.

*REPORT ::= exito | claseLlena | yaInscrito | noInscrito |
yaAprobado | noAprobado*

Esquema de exito

- El esquema Exito tiene exactamente una declaracion y un predicado. Este sera combinado con otros esquemas para indicar sus salidas exitosas.

Exito

report! : REPORT

report! = exito

Esquemas de errores

ClaseLlena

∃Clase

report! : REPORT

#inscrito ≥ maximoTamañoClase

report! = claseLlena

La clase ya está completa

YaInscrito

∃Clase

alumno? : ALUMNO

report! : REPORT

alumno? ∈ inscrito

report! = yaInscrito

Un alumno no puede ser inscrito de nuevo si ya lo estaba

NoInscrito

∃Clase

alumno? : ALUMNO

report! : REPORT

alumno? ∉ inscrito

report! = noInscrito

Si un alumno no está inscrito no puede ser aprobado

Esquemas de errores

YaAprobado

*$\exists$ Clase
alumno?: ALUMNO
report!: REPORT*

*alumno? $\in$ aprobado
report! = yaAprobado*

Un alumno que está aprobado
no puede ser aprobado nuevamente

NoAprobado

*$\exists$ Clase
alumno?: ALUMNO
report!: REPORT*

*alumno? $\notin$ aprobado
report! = noAprobado*

Si un alumno que no está aprobado
No puede obtener el certificado

Calculo de Esquema

- Usamos la conjuncion ($\wedge$), para combinar dos esquemas, y la disyuncion ($\vee$) para representar alternativas.

- Ejemplos:

$BienInscrito \wedge Exito$
 $(BienInscrito \wedge Exito) \vee ClaseLlena \vee YaInscrito$

- Version completa de los procesos:

$Inscripcion \triangleq (BienInscrito \wedge Exito) \vee ClaseLlena \vee YaInscrito$
 $Completacion \triangleq (BienCompletado \wedge Exito) \vee NoInscripto \vee YaAprobado$
 $ObtencionCertificado \triangleq (ObtenerCertificado \wedge Exito) \vee NoAprobado$

Contenido

- Descripción
- Conjuntos
- Tipos Basicos
- El Tipo Entero
- Predicados
- Esquemas
- Tipos de Conjuntos
- Operaciones entre Conjuntos
- Schema Calculus
- **Relaciones Binarias**

Relaciones Binarias

- Sean A y B conjuntos una relacion binaria R es un subconjunto del conjunto $A \times B$. El conjunto A se denomina el dominio de la relacion y B se denomina el rango de la relacion.
- Ejemplo:

• $ALUMNO = \{juan, maria, jose\}$
• $CURSO = \{cálculo, álgebra, programación\}$
• $R = \{(juan, cálculo), (juan, álgebra), (maria, álgebra), (jose, programación)\}$ es una relacion pues $R \subseteq ALUMNO \times CURSO$

Como una relacion es un conjunto, entonces son validas todas las operaciones sobre conjunto previamente definidas.

Notacion “maplet”

- Un elemento de una relación es un par ordenado, por ejemplo $(juan, \text{cálculo})$. Este par ordenado puede ser escrito como: $juan \mapsto \text{cálculo}$. Esta es conocida como la **notación maplet**, y $juan \mapsto \text{cálculo}$ es conocido como un *maplet*.

- $(juan, \text{cálculo}) = juan \mapsto \text{cálculo}$

- Usamos la notación maplet para representar un elemento en una relación binaria, usamos la notación de par ordenado para representar un par en particular. Por ejemplo:

- $parejas = \{juan \mapsto maria, ana \mapsto pedro\}$

- $unaPareja = (juan, maria)$

Algunas funciones en Z

- $first(juan, maría) = juan$
- $second(juan, maría) = maría$
- $dom\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\} = \{1, 2, 3\} \rightarrow$ dominio de la relación
- $ran\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\} = \{a, b, c\} \rightarrow$ rango de la relación

Declaracion de una Relacion Binaria

- Supongamos que hemos definido los tipos del conjunto de todas las fechas y el conjunto de todas las personas. $[FECHA, PERSONA]$

- Definimos cita como la relacion binaria desde *FECHA* a *PERSONA* como:

$cita: FECHA \mapsto PERSONA$
$cita = \{7nov \mapsto tomas, 7nov \mapsto ana, 8nov \mapsto jose\}$

- El signo doble flecha es el operador relacion binaria

Restricciones al Dominio

- Supongamos que tenemos la relación binaria
 - $\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$
 - Su dominio es $\{1, 2, 3\}$
 - Su rango es $\{a, b, c\}$
- Si queremos restringir la relación de modo que el dominio sea $\{2\}$, tendríamos $\{2 \mapsto b\}$ y escribimos:
 - $\{2\} \triangleleft \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\} = \{2 \mapsto b\}$
 - Donde $\triangleleft$ es el *operador restricción de dominio*

Restricciones al Rango

- Supongamos que tenemos la relación binaria
 - $\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$
 - Su dominio es $\{1, 2, 3\}$
 - Su rango es $\{a, b, c\}$
- Si queremos restringir la relación de modo que el rango sea $\{b\}$, tendríamos $\{2 \mapsto b\}$ y escribimos:
 - ✱ $\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\} \triangleright \{b\} = \{2 \mapsto b\}$
 - ✱ Donde $\triangleright$ es el *operador restricción de rango*

Relacion Inversa

- Supongamos que tenemos el tipo *BEBIDA*, el conjunto de todas las bebidas. Definimos **costos** como una relación binaria desde las bebidas al precio.

$$costos: BEBIDA \mapsto \mathbb{Z}$$
$$costos = \{te \mapsto 150, caf\acute{e} \mapsto 250, chocolate \mapsto 500, sopa \mapsto 600\}$$

- Podemos definir la relación inversa que se define desde el precio hasta las bebidas

$$compras: \mathbb{Z} \mapsto BEBIDA$$
$$compras = \{150 \mapsto t\acute{e}, 250 \mapsto caf\acute{e}, 500 \mapsto chocolate, 600 \mapsto sopa\}$$

- Se escribe $costos \sim = compras$

Compuesta

- Supongamos que tenemos los tipos $[PERSONA, OFICINA]$, el conjunto de todas las personas y oficinas respectivamente. Definamos las relaciones binarias:

$$tieneAnexo: PERSONA \mapsto \mathbb{Z}$$
$$tieneAnexo = \{juan \mapsto 317, tomás \mapsto 208, jimy \mapsto 326, maría \mapsto 225\}$$
$$anexoOficina: \mathbb{Z} \mapsto OFICINA$$
$$anexoOficina = \{317 \mapsto A306, 208 \mapsto A39, 326 \mapsto A306, 225 \mapsto A39\}$$

- Podemos relacionar las personas con sus oficinas partiendo desde $PERSONA$ llegamos a $OFICINA$ vía $\mathbb{Z}$. Por ejemplo, *juan* tiene el anexo 317 que está en la oficina A306, con lo cual concluimos que *juan* tiene la oficina A306.

$$tieneAnexo; anexoOficina = \{juan \mapsto A306, tomás \mapsto A39, jimy \mapsto A306, maría \mapsto A39\}$$

Imagen Relacional

- Supongamos que tenemos la relación binaria
 - $\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$
 - Su dominio es $\{1, 2, 3\}$ - Su rango es $\{a, b, c\}$
- Deseamos responder a la pregunta ¿cuál es la segunda coordenada del par cuya primera componente es 2?. La respuesta es b , escribimos:
 - $\{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}[\{2\}] = \{b\}$
 - Donde $[\]$ es el **operador imagen relacional**
- Ejemplo: Si $R = \{(1, a), (2, b), (3, c), (4, d), (5, e)\}$ y $S = 2..4$ evalúe:
 - $R[S]$
 - $\text{ran}(S \curvearrowright R)$